TP de Programmation Orientée Objet
Licence Informatique *

Semaine du 15 septembre 2025

Ce sujet de TP reprend très largement le premier chapitre du livre “Conception objet en Java avec BlueJ” de David Barnes et Michael Kölling (Editeur Pearson Education).

1 Création d’objets

Télécharger le “Projet BlueJ pour le TP” (i.e. fichier archive de nom `shapesfrBase.tar.gz`) sur le site internet du cours (accessible à partir de l’ENT) et en extraire le contenu à l’aide de la commande :
`tar -xvzf shapesfrBase.tar.gz`
puis placez vous dans le répertoire nouvellement créé et nommé `shapesfrBase` à l’aide de la commande :
`cd shapesfrBase`

Lancez BlueJ à l’aide de la commande : `bluej` et ouvrez le projet `shapesfrBase` que vous venez de récupérer. La fenêtre de la figure 1 apparait.

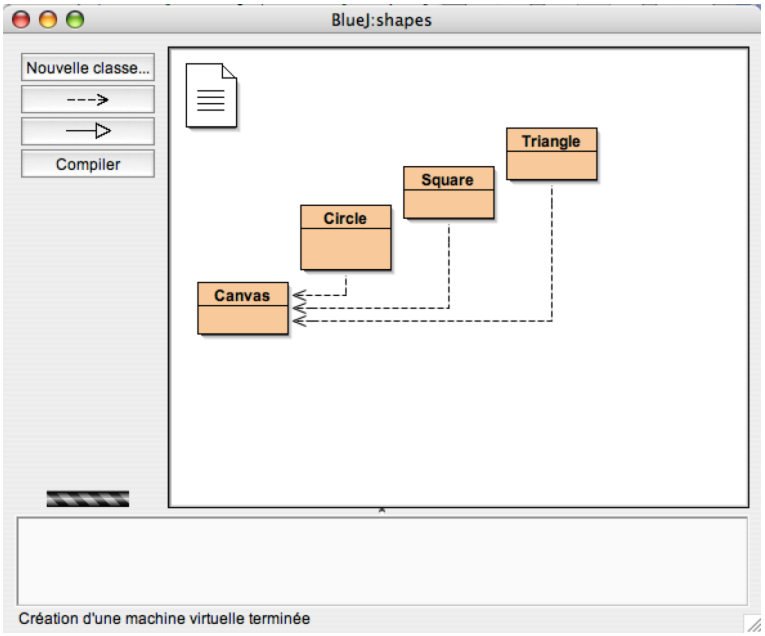

FIGURE 1 – Le projet shapes dans BlueJ

La fenêtre contient un diagramme. Chacun des rectangles colorés représente une classe du projet : `Circle`, `Square`, `Triangle` et `Canvas`.

Cliquez du bouton droit sur la classe `Circle`. Dans le menu déroulant qui s’affiche, choisissez :

```
new Circle()
```

Pour attribuer à l’instance le nom qui s’inscrit par défaut, cliquez su OK. Un rectangle rouge apparait au bas de l’écran, il est intitulé “circle_1” (cf. figure 2).

Vous venez de créer votre premier objet. L’icône rectangulaire de la figure 1, intitulée “Circle”, représente la *classe* `Circle` alors que `circle_1` représente un objet créé à partir de cette classe (on parle aussi d’une *instance* de cette

*M. Champesme – Département d’Informatique – Institut Galilée

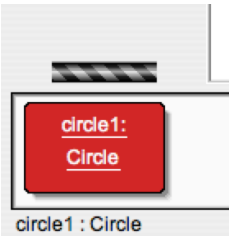

FIGURE 2 – Un objet sur le bureau des objets.

classe). La zone située au bas de l'écran et contenant l'objet s'intitule le *bureau des objets* (ou "object bench" en anglais).

Créez un autre cercle. Puis créez un carré.

2 Appel des méthodes

Cliquez du bouton droit sur l'un des objets cercle (et non sur la classe !). Un menu contextuel s'affiche, contenant diverses options. Choisissez `makeVisible` pour dessiner une représentation de ce cercle dans une nouvelle fenêtre (cf. figure 3).

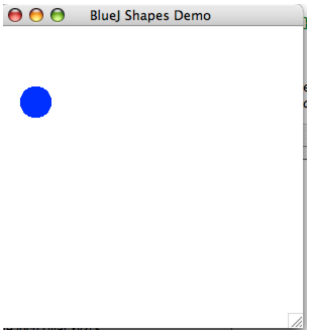

FIGURE 3 – Le dessin d'un cercle.

Le menu du cercle contient plusieurs autres opérations. Vous pouvez tester `moveRight` et `moveDown` à plusieurs reprises pour rapprocher le cercle du centre de la fenêtre. Essayez également `makeInvisible` et `makeVisible` pour masquer et afficher le cercle.

Les choix du menu du cercle permettent de manipuler celui-ci. En langage Java, on les désigne sous le nom de *méthodes*. On dit communément que l'on *appelle* ou que l'on *invoque* ces méthodes.

3 Paramètres

Invoquez maintenant la méthode `moveHorizontal`. Une boîte de dialogue vous demande de saisir certaines informations (cf. figure 4). Tapez 50 et cliquez sur OK. Le cercle se déplacera de 50 pixels sur la droite.

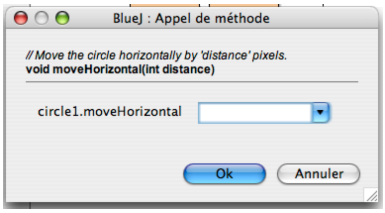

FIGURE 4 – Boîte de dialogue pour l'appel d'une méthode avec un paramètre.

Invoquez les méthodes `moveVertical`, `slowMoveVertical` et `changeSize`. Essayez de déplacer le cercle de 70 pixels sur la gauche en utilisant `moveHorizontal`.

4 Instances multiples

Créez plusieurs objets cercle sur votre bureau des objets. Pour cela, sélectionnez `new Circle()` dans le menu déroulant de la classe `Circle`. Affichez les, puis déplacez les à l'écran à l'aide des méthodes `move`. Augmentez la taille de l'un d'entre eux, que vous transformerez en jaune, réduisez en un autre, que vous rendrez vert. Essayez également les autres formes : créez quelques triangles et quelques carrés. Modifiez leur positions leurs tailles et leurs couleurs.

Une fois la classe créée, vous pouvez construire autant d'objets (ou d'instances) de cette classe que vous le souhaitez. A partir de la classe `Circle` vous pouvez créez de nombreux cercles. Idem avec les carrés à partir de la classe `Square`.

Chacun de ces objets possède sa propre position, sa couleur, sa taille. Pour modifier l'attribut d'un objet (comme sa taille), appelez une méthode sur cet objet. Cela affectera l'objet concerné et non les autres.

5 État d'un objet

L'ensemble des valeurs de tous les attributs qui définissent un objet (comme la position x, la position y, la couleur, le diamètre et l'état de visibilité d'un cercle) est également appelé l'*état* d'un objet.

Dans BlueJ, pour connaître l'état d'un objet, sélectionnez la fonction "Inspecter" dans le menu déroulant de cet objet. Lorsqu'on inspecte un objet, une fenêtre semblable à celle présentée à la figure 5 s'affiche. C'est ce que l'on appelle l'*inspecteur d'objet*.

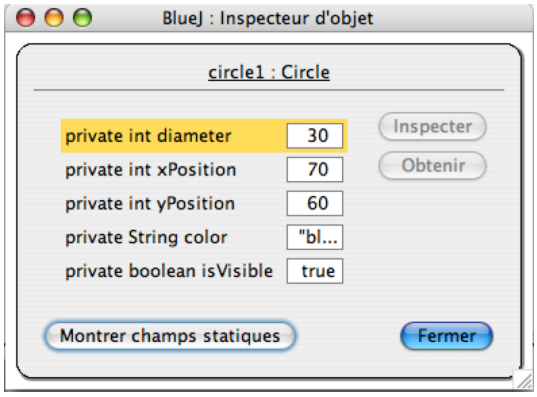

FIGURE 5 – L'inspecteur d'objets.

Assurez-vous que le bureau des objets contient plusieurs objets, puis étudiez les tour à tour. Essayez de modifier l'état de l'un d'entre eux (par exemple en appelant la méthode `moveLeft`) lorsque l'inspecteur d'objets est ouvert. Remarquez que seules les valeurs de cet objet changent dans l'inspecteur.

6 Définition d'un objet

En étudiant les différents objets, on remarque que les objets d'une même classe disposent tous des mêmes champs. En fait le nombre, le type et les noms des champs sont identiques, mais la valeur effective d'un champ donné peut différer pour chaque objet. Au contraire, les objets de classes différentes peuvent avoir des champs différents. Un cercle, par exemple, dispose d'un champ "diamètre" (i.e. `diameter`) tandis qu'un triangle possède des champs de largeur (`width`) et de hauteur (`height`).

En fait, les types et les noms des champs se définissent dans une classe, et non dans un objet. Ainsi, la classe `Circle` impose que chaque objet cercle (i.e. chaque instance de la classe `Circle`) dispose de cinq champs intitulés `diameter`, `xPosition`, `yPosition`, `color` et `isVisible`. Elle définit également les types de ces champs. En effet, les trois premiers sont du type `int`, tandis que la couleur est du type `String` et que `isVisible` est du type `boolean`.

Lorsqu'on crée une instance d'une classe (par exemple `Circle`), il contient automatiquement ces champs. Leurs valeurs sont alors stockées dans l'instance. Ainsi, chaque cercle pourra avoir une couleur différente des autres cercles (cf. figure 6).

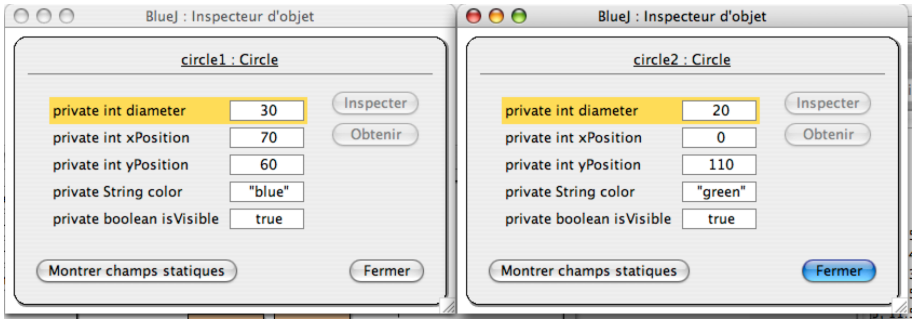

FIGURE 6 – Champs de deux instances d'une même classe (la classe `Circle`).

La situation est identique pour les méthodes. Elles sont définies dans la classe de l'objet. En conséquence, tous les objets d'une classe donnée disposent des mêmes méthodes. Toutefois, les méthodes sont toujours invoquées sur un objet spécifique. Cela permet de connaître avec certitude l'objet modifié par un appel à la méthode `moveRight`.

Utilisez les formes du projet shapes pour dessiner une maison et un soleil, à la manière de ceux qui sont représentés à la figure 7. Ecrivez en même temps ce que vous devez faire pour y parvenir.

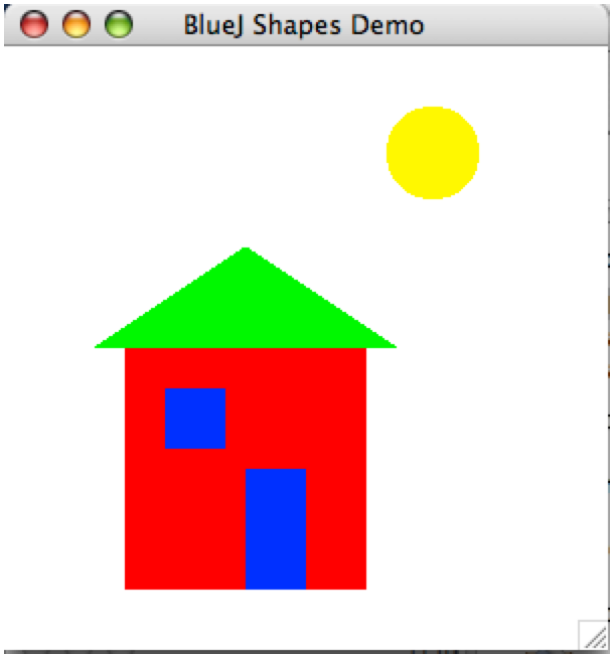

FIGURE 7 – Une image créée à partir d'un ensembles d'objets formes.

En vous inspirant de la classe `Square` qui modélise un carré, écrivez le code source d'une nouvelle classe nommée `RectangleShape` (*ne pas utiliser le nom `Rectangle`*, car il est déjà pris par une autre classe) modélisant la forme géométrique rectangle. Dans un deuxième temps définnisez une nouvelle classe `House` permettant de représenter une maison constituée d'un rectangle (i.e. une instance de la classe `RectangleShape`) et d'un triangle (i.e. une instance de la classe `Triangle`).